AI应用开发工程师——比特鹰成长计划

> 本资料中所有知识点均可结合 AI 进行更深入的学习，所有附带链接仅供学习参考。

LLM 基础

LLM 基础是 AI 应用开发的底座。

学习大模型不能只停留在“会调 API”，而要理解模型的基本生成方式、Token、上下文窗口、模型类型、API 协议、流式响应、结构化输出、工具 调用和 Embedding 等核心概念。

掌握这些基础后，学习 Prompt、RAG、Agent、Workflow 才不会停留在框架调用层。

大模型基本原理

Transformer 基础、自回归生成（Next Token Prediction）、预训练、指令微调、RLHF / RLAIF、上下文学习（In-Context Learning）。

https://jalammar.github.io/illustrated-transformer/

Token 与上下文窗口（关键）

Tokenization、输入 Token、输出 Token、Context Window、最大输入长度、最大输出长度、上下文溢出、长上下文的“丢中间”（Lost in the Middle）问题。

https://help.openai.com/en/articles/4936856-what-are-tokens-and-how-to-count-them

---

采样参数

Temperature、Top-p、Top-k、Frequency Penalty、Presence Penalty、Seed、Stop Sequences，理解确定性、创造性、重复控制和输出终止的取舍。

https://docs.vllm.ai/en/v0.6.4/dev/sampling_params.html

模型类型差异

普通 Chat Model、Reasoning Model、多模态模型、Embedding Model、Rerank Model、语音/实时模型，理解不同模型适合的任务边界。

API 协议模型（关键）

Chat Completions / Responses / Messages API 的基本抽象，理解 message、role、content、system / user / assistant / tool message 的职责。

https://platform.openai.com/docs/api-reference/chat/create
https://platform.openai.com/docs/api-reference/responses/create
https://platform.claude.com/docs/en/build-with-claude/working-with-messages

---

请求与响应结构（关键）

model、messages/input、instructions/system prompt、tools、tool_choice、response_format、stream、usage、finish_reason 等常见字段。

https://platform.openai.com/docs/api-reference/chat/create
https://platform.openai.com/docs/api-reference/responses

https://docs.anthropic.com/en/api/handling-stop-reasons

Streaming 流式响应（关键）

SSE / event stream、delta、增量文本、工具调用流式事件、流式中断、前端打字机效果。

https://platform.openai.com/docs/guides/streaming-responses
https://platform.openai.com/docs/api-reference/responses-streaming
https://docs.anthropic.com/en/docs/build-with-claude/streaming

---

结构化输出（关键）

JSON Mode、JSON Schema、Structured Outputs、字段约束、枚举、嵌套对象、结构化输出失败后的修复策略。

https://json-schema.org/understanding-json-schema/basics
https://platform.openai.com/docs/guides/structured-outputs
https://openai.com/index/introducing-structured-outputs-in-the-api/

Function Calling / Tool Use（关键）

模型只是生成工具调用意图，真正执行工具的是业务系统；需要理解工具描述、参数 Schema、tool_call_id、tool result、并行工具调用和工具错误返回。

https://platform.openai.com/docs/guides/function-calling
https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview

---

多轮对话状态

大模型 API 调用通常是无状态的，需要业务侧维护历史消息、摘要、短期记忆、长期记忆、上下文压缩和会话状态。

https://platform.openai.com/docs/guides/conversation-state
https://platform.claude.com/docs/en/build-with-claude/mid-conversation-system-messages

---

Embedding 基础

文本向量、语义相似度、向量维度、归一化、Cosine / Dot Product / L2、Embedding 模型选型、批量生成和向量缓存。

https://platform.openai.com/docs/guides/embeddings

项目实战：大模型 API Playground

从 0 到 1 实现一个大模型 API 调用 Playground，用于理解 LLM API 的核心能力。

要求实现：

- 支持普通对话调用
- 支持流式响应
- 支持结构化输出
- 支持 Function Calling / Tool Use
- 支持 Embedding 生成与相似度计算
- 记录每次调用的模型、输入、输出、Token 用量和耗时
- 对比至少两个模型或两个供应商的 API 差异

验收标准：

- 能清楚解释 message、role、tool call、tool result、stream、usage 的含义
- 能演示一次完整工具调用链路
- 能演示结构化输出失败时如何处理
- 仓库包含 README、运行说明、示例请求和测试用例

请创建一个 GitHub 仓库存放所有代码、文档等，最后提交**仓库链接**即可。

Prompt Engineering

提示词工程是控制大模型行为的基础能力。

学习 Prompt 不只是学习“怎么问”，而是学习如何组织任务、上下文、约束、示例和输出格式，让模型在复杂场景下稳定地产生可用结果。

提示词结构（关键）

角色设定（Role/System Prompt）、任务描述（Instruction）、上下文（Context）、输入数据（Input）、输出格式（Output Format）、约束条件（Constraints）。

https://platform.openai.com/docs/guides/prompt-engineering
https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview

---

Zero-shot / Few-shot

Zero-shot 是无示例直接完成任务，Few-shot 是通过少量示例约束模型输出格式、判断标准和表达风格。

https://www.promptingguide.ai/zh/techniques/fewshot

Chain-of-Thought（CoT）

通过分解问题、逐步推理、先分析后回答，提升模型在复杂推理、判断、规划类任务中的稳定性。

https://www.promptingguide.ai/zh/techniques/cot

ReAct

Reasoning + Acting，让模型在推理过程中决定是否调用工具、观察工具结果，再继续完成任务，是 Agent 工具调用的基础范式。

https://www.promptingguide.ai/zh/techniques/react

Self-Critique / Reflexion

让模型对自己的输出进行检查、反思和修正，适用于代码生成、报告生成、复杂任务复核等场景。

https://www.promptingguide.ai/zh/techniques/reflexion

实践要求

Prompt Engineering 不单独设置项目实战，这是任何一个 AI 项目的基础，需要根据具体项目情况定义，请在后续项目中体现。

要求在每个项目 README 中说明：

- 使用了哪些 Prompt 结构
- 为什么这样设计 System Prompt 和约束条件
- 是否使用 Few-shot 示例
- 如何控制输出格式
- 如何处理模型不确定、越界回答或格式错误
- 至少记录 3 个 Prompt 修改前后的效果对比

RAG

RAG 是把外部知识引入大模型的核心方式，用于解决知识过时、私有知识不可见、模型幻觉等问题。

学习 RAG 的重点不是“向量库 + 大模型”这么简单，而是理解文档处理、索引、召回、重排、上下文拼接、引用溯源和效果评估的完整链路。

文档加载与解析

PDF、Word、HTML、Markdown、代码、表格、图片 OCR、多模态文档解析，理解不同文档结构对检索质量的影响。

https://docs.llamaindex.ai/en/stable/module_guides/loading/

分块策略（关键）

固定长度分块、递归分块、语义分块、基于 Markdown / Code 结构分块、父子分块、滑动窗口与 overlap。

https://www.pinecone.io/learn/chunking-strategies/
https://docs.llamaindex.ai/en/stable/module_guides/loading/node_parsers/

Embedding 与向量检索（关键）

Embedding 模型选型、向量维度、归一化、Cosine / Dot Product / L2、Top-K、相似度阈值、向量缓存。

https://platform.openai.com/docs/guides/embeddings

向量数据库与索引

FAISS、Milvus、Qdrant、Weaviate、Chroma、Pinecone、PGVector、Redis Stack，以及 FLAT、IVF、HNSW、PQ 等索引算法。

https://www.pinecone.io/learn/vector-database/

元数据过滤与溯源

标题、章节、来源、时间戳、用户权限、文档版本等元数据抽取；理解先过滤后检索、先检索后过滤的取舍。

https://docs.llamaindex.ai/en/stable/module_guides/indexing/metadata_extraction/

混合检索与多路召回

关键词检索、向量检索、BM25、Hybrid Search、RRF 融合、多查询改写，用于提升召回覆盖率。

https://www.pinecone.io/learn/hybrid-search/

Rerank 重排序（关键）

Cross-Encoder、Bi-Encoder、bge-reranker、Cohere Rerank、Jina Rerank，理解粗排 + 精排二阶段检索架构。

https://docs.cohere.com/docs/reranking

生成阶段优化

上下文拼接、上下文压缩、引用与溯源、不确定时拒绝回答、避免 Lost in the Middle，控制模型只基于检索内容回答。

https://www.pinecone.io/learn/retrieval-augmented-generation/

RAG 评估（关键）

评估检索准确率、召回率、答案忠实度、上下文相关性、幻觉率，使用测试集持续验证 RAG 效果。

https://docs.ragas.io/

Advanced RAG

Query Rewrite、HyDE、Self-Query、Agentic RAG、GraphRAG，用于解决复杂查询、跨文档推理和多轮检索问题。

https://github.com/microsoft/graphrag

项目实战：RAG + Tool Calling Agent

从 0 到 1 实现一个可以读取文档、检索知识、调用工具并回答问题的 Agent 应用。

要求实现：

- 支持上传或导入 Markdown / PDF 文档
- 实现文档切分、Embedding、向量检索
- 支持引用来源和不确定时拒绝回答
- 至少实现 2 个工具，例如网页搜索、计算器、文件查询、时间查询
- Agent 能根据问题决定是否检索、是否调用工具
- 记录 Agent 每一步的 reasoning 摘要、工具调用和最终结果
- 准备不少于 20 条测试问题用于评估效果

验收标准：

- 能解释 RAG 的完整链路
- 能解释工具调用和普通函数调用的区别
- 能提供检索效果、回答质量和失败 case 分析

请创建一个 GitHub 仓库存放所有代码、文档等，最后提交**仓库链接**即可。

Agent 应用

Agent 应用是在 LLM 基础之上构建可执行、可检索、可调用工具、可管理上下文的 AI 系统。

学习重点不是绑定某个框架，而是理解 Prompt、RAG、工具调用、工作流、状态管理、记忆、评估与安全这些核心能力如何组合成可靠的 AI 应用。

Agent 基本概念

理解 Agent、Chatbot、Workflow 的边界：Chatbot 偏对话，Workflow 偏确定流程，Agent 偏动态决策和工具调用。

https://arthurchiao.art/blog/ai-agent-white-paper-zh/#2-%E4%BB%80%E4%B9%88%E6%98%AF-agent

---

Agent 核心循环（关键）

理解模型如何根据用户目标、上下文和工具结果持续推进任务。

https://code.claude.com/docs/en/agent-sdk/agent-loop

工具调用设计（关键）

工具描述、参数 Schema、工具选择、工具结果格式、工具错误返回、并行工具调用、工具超时与重试。

https://platform.openai.com/docs/guides/function-calling

Workflow 模式

Prompt Chaining、Routing、Parallelization、Orchestrator-Workers、Evaluator-Optimizer，理解可控流程优先于盲目 Agent 化。

https://www.anthropic.com/engineering/building-effective-agents

规划与任务拆解

Plan-and-Execute、动态重规划、子任务拆分、任务依赖、终止条件，理解复杂任务如何从目标变成可执行步骤。

https://lilianweng.github.io/posts/2023-06-23-agent/
https://www.langchain.com/blog/planning-agents

状态管理（关键）

全局状态、局部状态、消息历史、工具调用记录、任务进度、状态持久化、断点恢复，避免 Agent 变成不可复盘的黑盒。

https://www.inngest.com/blog/durable-execution-key-to-harnessing-ai-agents
https://langchain-ai.github.io/langgraph/concepts/low_level/
https://docs.langchain.com/oss/python/langgraph/persistence

记忆系统（关键）

短期记忆、长期记忆、情景记忆、语义记忆、记忆提取、记忆压缩、遗忘机制，理解记忆和上下文的区别。

https://www.ibm.com/think/topics/ai-agent-memory
https://lilianweng.github.io/posts/2023-06-23-agent/#component-two-memory

---

上下文工程（关键）

上下文选择、上下文裁剪、上下文压缩、动态注入、工具结果摘要、历史消息摘要，控制模型在有限窗口内获得正确信息。

https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

---

MCP

理解 Model Context Protocol 的 Server / Client / Tools / Resources / Prompts 架构，以及它如何标准化模型与外部工具、数据源的连接方式。

https://modelcontextprotocol.io/docs/getting-started/intro

Human-in-the-Loop

审批节点、中断与恢复、人工修正、敏感操作确认，避免 Agent 在高风险任务中自动执行不可逆操作

https://langchain-ai.github.io/langgraph/concepts/human_in_the_loop/

Agent 评估（关键）

评估任务完成率、工具调用准确率、步骤合理性、失败恢复能力、成本、延迟和多轮稳定性。

https://langfuse.com/docs/evaluation/overview
https://docs.langchain.com/langsmith/evaluation

Agent 安全（关键）

Prompt Injection、防止工具越权、工具权限控制、沙箱执行、敏感数据保护、外部内容不可信、关键操作二次确认。

https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html

项目实战：从 0 到 1 写一个 Claude Code

认真学习这个开源项目源码，了解如何从 0 到 1 构建一个 Claude Code。

把所有代码读完做到了如指掌。

https://github.com/shareAI-lab/learn-claude-code

请创建一个 GitHub 仓库存放所有代码、文档等，最后提交**仓库链接**即可。

后端工程

后端工程是 AI 应用落地的基础。

学习重点不是堆技术栈，而是掌握异步编程、接口开发、数据建模、数据库访问、缓存、消息队列、容器化、测试和自动化交付这些核心能力。

这样才能让 AI Agent、RAG、工具调用等能力可以稳定运行在真实服务中。

Python Async

Python 异步编程是 AI 后端的基础能力。

需要理解 asyncio、协程、事件循环、任务调度、并发控制、异步 I/O、超时取消，以及什么时候该用 async、什么时候不该用 async。

https://docs.python.org/3/library/asyncio.html

FastAPI

FastAPI 是 Python 后端常用的异步 Web 框架。

需要掌握路由、请求响应模型、依赖注入 Depends、中间件、异常处理、生命周期管理、OpenAPI 文档和异步接口开发。

https://fastapi.tiangolo.com/zh/learn/

Pydantic

Pydantic 是 Python 数据校验和序列化的核心工具。

需要掌握 BaseModel、Field、类型标注、嵌套模型、枚举、校验器、配置项、DTO 设计，以及它在 API 入参、出参和结构化数据建模中的作用。

https://docs.pydantic.dev/latest/

SQLAlchemy & Alembic

SQLAlchemy 是 Python 主流 ORM，Alembic 用于数据库迁移。

需要掌握 SQLAlchemy 2.0、AsyncSession、模型定义、关系映射、事务管理、查询构造、连接池，以及数据库Schema 迁移。

https://docs.sqlalchemy.org/en/20/orm/extensions/asyncio.html
https://alembic.sqlalchemy.org/en/latest/

PostgreSQL

PostgreSQL 是后端常用关系型数据库，也是很多 AI 应用存储业务数据、会话数据、任务状态和向量数据的基础。

需要掌握表设计、索引、事务、锁、JSONB、全文检索，以及 pgvector 的基本使用。

https://www.postgresql.org/docs/
https://github.com/pgvector/pgvector

Redis

Redis 是高性能内存数据库，常用于缓存、限流、分布式锁、会话状态、任务状态和轻量消息流。

需要掌握 String、Hash、Set、ZSet、Stream、TTL、Pipeline、Lua、Pub/Sub、RedisJSON 和 Vector Search。

https://redis.io/docs/latest/
https://redis.io/docs/latest/develop/interact/search-and-query/

RabbitMQ

RabbitMQ 是基于 AMQP 的消息队列，适合异步任务、服务解耦、流量削峰和后台任务处理。

需要掌握 Producer、Consumer、Exchange、Queue、Binding、Routing Key、Ack、Prefetch、死信队列和 Quorum Queue。

https://www.rabbitmq.com/tutorials
https://www.rabbitmq.com/docs

Testing

测试是后端工程质量的基本保障。

需要掌握单元测试、集成测试、异步测试、Mock、Fixture、测试数据库、接口测试，以及如何为外部 API、消息队列和数据库访问编写可靠测试。

https://docs.pytest.org/en/stable/
https://pytest-asyncio.readthedocs.io/en/latest/

Docker

Docker 是服务容器化的基础。

需要掌握 Dockerfile、镜像构建、容器运行、环境变量、volume、network、multi-stage build、docker compose，以及本地开发环境编排。

https://docs.docker.com/get-started/
https://docs.docker.com/compose/

Kubernetes 基础

Kubernetes 是服务部署和编排的基础。

需要掌握 Pod、Deployment、Service、Ingress、ConfigMap、Secret、Job、CronJob、HPA、健康检查和滚动发布。

https://kubernetes.io/docs/concepts/

Microservices

微服务是围绕业务能力拆分服务的架构方式。

需要理解单体 vs 微服务、DDD 边界、同步通信、异步通信、API 网关、配置管理、服务治理、可观测性，以及什么时候不应该拆微服务。

https://microservices.io/

CI/CD

CI/CD 用于自动化检查、测试、构建和部署。

需要掌握 GitHub Actions、workflow、job、step、trigger、secrets、cache、artifact、测试流水线、Docker 镜像构建和多环境部署。

https://docs.github.com/en/actions

项目实战：AI 服务后端系统

从 0 到 1 实现一个可运行的 AI 后端服务，用于验证 Python 后端工程能力。

要求实现：

- 使用 FastAPI 提供 REST API
- 使用 Pydantic 定义请求和响应模型
- 使用 PostgreSQL 保存用户、会话、消息和任务记录
- 使用 Redis 做缓存或任务状态存储
- 使用 RabbitMQ 或后台任务队列处理异步任务
- 使用 Docker Compose 启动完整本地环境
- 编写单元测试和接口测试
- 提供基础 CI 流水线，至少包含 lint 和 test

验收标准：

- 服务可以一条命令启动
- API 文档可访问
- 测试可以一条命令运行
- 能解释数据库表设计、异步任务设计和错误处理方式
- README 包含架构说明、启动方式、接口示例和测试说明

请创建一个 GitHub 仓库存放所有代码、文档等，最后提交**仓库链接**即可。

综合实战

从 0 到 1 写一个 OpenClaw

认真学习这个开源项目源码，了解如何从 0 到 1 构建一个 OpenClaw。

把所有代码读完做到了如指掌。

https://nanoclaw.dev/

请创建一个 GitHub 仓库存放所有代码、文档等，最后提交**仓库链接**即可。